Remediation Playbook for Defender Security Optimization Assessments

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Remediation Playbook for Defender Security Optimization Assessments 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Boundary . . . 5

Effort . . . 6

1. Solution Overview — Remediation Playbook for Defender Security Optimization Assessments 6

What this playbook is . . . 6

Why a separate, gated playbook . . . 6

The remediation model (per finding) . . . 7

What is in scope . . . 7

What is out of scope (and where it lives) . . . 7

Connections and authentication used . . . 7

The standard deployment model . . . 8

2. Requirements — Remediation Playbook for Defender Security Optimization Assessments 8

R1. A completed assessment findings report to remediate . . . 8

R2. Production change-approval process and a named change window . . . 8

R3. A named pilot scope with rollback / break-glass . . . 8

R4. Least-privilege read roles for ingest/verify/re-score . . . 9

R5. A scoped admin role for the apply step ONLY . . . 9

R6. Write scopes for the specific remediation (only where required) . . . 9

R7. A secure location for the evidence report . . . 9

Global Requirements — Assessments (category-level) 9

G1. The workload/tenant being assessed . . . 10

G2. Read-only assessor access (least privilege) . . . 10

G3. Microsoft Graph / Az / management API access . . . 10

G4. Licensing for the products in scope . . . 10

G5. Scope definition and a point-in-time baseline . . . 10

G6. Best-practice reference baseline . . . 11

G7. Output handling and confidentiality . . . 11

G8. Remediation ownership (for the paired remediation) . . . 11

3. Implementation — Remediation Playbook for Defender Security Optimization Assessments 11

The finding-to-remediation map . . . 12

Stage 1 — Assess (ingest the finding and record the plan) . . . 13
Stage 2 — Configure (plan the fix and the pilot) . . . 13
Stage 3 — Pilot (apply to the pilot scope, gated) . . . 13
Stage 4 — Validate (verify the control, then re-score) . . . 14
Stage 5 — Expand (stage the rollout and evidence it) . . . 14
5. Testing — Remediation Playbook for Defender Security Optimization Assessments 14
Test 1 — Ingest is read-only and maps findings . . . 15
Test 2 — Preview (`-WhatIf`) changes nothing . . . 15
Test 3 — Apply requires the explicit switch (and disruptive confirmation) . . . 15
Test 4 — Apply is scoped to the pilot only . . . 15
Test 5 — Verification confirms the specific control . . . 15
Test 6 — Re-score proves closure . . . 15
Test 7 — Rollback works . . . 15
Test 8 — Evidence report is complete and traceable . . . 16
Exit criteria . . . 16
6. Troubleshooting — Remediation Playbook for Defender Security Optimization Assessments 16
`SOAREM-Discovery.ps1` can't map a finding . . . 16
`-WhatIf` seems to make a change . . . 16
Apply fails with an authorization error . . . 16
The remediation refuses to enforce . . . 16
The change hit more than the pilot scope . . . 17
Verification says the control isn't in place . . . 17
Re-score doesn't move after a verified change . . . 17
Rollback leaves a partial state . . . 17
Locked out after a tightening change . . . 17
7. Best Practices — Remediation Playbook for Defender Security Optimization Assessments 17
Preview before you touch anything . . . 17
Pilot before you enforce . . . 18
Change under approval, on the customer's authority . . . 18
Re-score is the proof, not the apply . . . 18
Keep read and write separated . . . 18
Track everything for audit . . . 18
Remediate by risk, in stages . . . 18
Know when to hand off to a deeper solution . . . 18
Where to go next (adjacent catalog areas) . . . 19
8. Deliverables Summary — Remediation Playbook for Defender Security Optimization Assessments 19

Documentation 19
PowerShell scripts (Microsoft Graph + Az) 19
What the buyer receives 20
Outcome 20
Appendix A — PowerShell Scripts 21

Executive Summary — Remediation Playbook for Defender Security Optimization Assessments

White-label · Assessments

An assessment that just produces a report gathers dust. The Security Optimization Assessments for the Defender line (Cloud Apps, Cloud, Endpoint, Identity, Office 365, and the unified Suite/XDR) hand you a **prioritized, risk-ranked findings backlog** — but a backlog is not a fix. This **Remediation Playbook** is the companion that **turns findings into closed gaps**: it maps every finding category across all five Defender products to a **pilot-first, `-WhatIf`/simulate-gated** remediation procedure, then **verifies** the fix and **re-scores** to prove the gap is closed — with change-tracking and an evidence report.

This package is written for an administrator with no prior remediation experience. Unlike the assessments (which are read-only), this playbook **makes changes** — so every change script defaults to a **safe preview (`-WhatIf` / simulation)**, is applied to a **pilot scope first**, and carries an explicit **rollback / break-glass** note. Nothing enforces until you have seen the preview and staged the rollout.

What you get

- A plain-language **Solution Overview** of the remediation model, the finding-to-remediation map across the five Defender products, and the gating/verification approach.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (category-wide prerequisites live once in the Global Requirements).
- An **Implementation Guide** of exact, ordered steps to remediate a finding end to end — pilot-first, preview-gated, verified, re-scored — following **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**.
- Five **PowerShell scripts** — a read-only findings ingest, a **`-WhatIf`-gated remediation** helper (safe preview by default), a post-change validation, a **re-score**, and a change-tracked evidence report.
- **Testing** and **Troubleshooting** guides so remediation is safe and repeatable.

What this delivers

The assessment backlog **actually closed**: each finding remediated on a pilot scope first with the change previewed before it is applied, then verified, then re-scored to prove closure — and an **evidence report** that shows what changed, when, by whom, and the before/after score.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — take the finding, configure the fix, apply it to a pilot in preview then for real, validate and re-score, then expand across scope with rollback ready.

Boundary

This playbook supplies the **technical remediation, verification, and re-score**, not the authorization to change production. **Production change-approval**, **risk-acceptance**, **remediation priority**, and any **regulatory / records / incident-response** determination are the customer's change-management, security, compliance, and SOC functions' decisions. Response actions and disruptive controls are

approval-gated. Licensing, plan, and preview facts are flagged **verify**.

Effort

A competent administrator can remediate a finding category on a pilot scope in a focused engagement, then stage the rollout. It is **repeatable** — the same pilot-first, preview-gated, re-score loop applies to every finding across every Defender product.

1. Solution Overview — Remediation Playbook for Defender Security Optimization Assessments

What this playbook is

This is the **remediation companion** to the Security Optimization Assessment (SOA) line for Microsoft Defender. The assessments are **read-only** — they enumerate configuration, score it against best practice, and produce a **prioritized, risk-ranked findings report**. This playbook takes that report and **closes the gaps**: for each finding category it provides a **pilot-first, preview-gated** remediation procedure, then **verifies** and **re-scores** to prove closure, with change-tracking and an evidence report.

It spans the finding categories across all five Defender products:

- **Defender for Cloud Apps** — connector/discovery coverage, app-governance/OAuth policies, session/access policies, alert hygiene.
- **Defender for Cloud** — Secure Score recommendations, plan coverage, regulatory-compliance mapping, agent/agentless coverage.
- **Defender for Endpoint** — onboarding/coverage, ASR/hardening, EDR settings, vulnerability remediation.
- **Defender for Identity** — sensor coverage, unresolved health alerts, recommended detections.
- **Defender for Office 365** — anti-phish/Safe Links/Safe Attachments policy hygiene, preset security policies.
- **Suite / XDR (cross-cutting)** — cross-pillar coverage, unified RBAC, incident correlation, suite Secure Score.

Why a separate, gated playbook

Remediation is where risk lives. The assessment could never touch the tenant — that is what made it safe to run. Fixing a finding, by contrast, **changes configuration**: enabling a plan costs money, tightening a Conditional Access / session policy can lock users out, activating unified RBAC changes who can do what, and a response action can disrupt a user or device. So remediation must be **disciplined**:

- **Preview first.** Every change script defaults to `-WhatIf` **/ simulation** — it shows exactly what *would* change and touches nothing until you pass an explicit apply switch.
- **Pilot first.** Apply to a small, named **pilot scope** (a pilot group, one subscription, a test policy) before any broad rollout.

• **Verify, then re-score.** After applying, confirm the specific control is now correct, then **re-run the relevant assessment score** to prove the finding is closed — the score is the objective proof.

• **Track and evidence.** Record what changed, when, by whom, and the before/after score, so the remediation is auditable.

• **Rollback ready.** Each remediation carries a rollback / break-glass note before it is applied.

The remediation model (per finding)

For each finding the playbook follows the same loop:

1. **Ingest** the finding from the assessment's findings report (read-only).
2. **Map** it to its remediation procedure (the finding-to-remediation map in Section 3).
3. **Preview** the change with `-WhatIf` / simulation on the pilot scope.
4. **Apply** to the pilot scope (explicit apply switch; approval-gated for disruptive/response actions).
5. **Verify** the specific control is now correct.
6. **Re-score** by re-running the relevant assessment score to prove closure.
7. **Expand** the change across scope in stages, with rollback ready.
8. **Evidence** — record the change and before/after score in the remediation evidence report.

What is in scope

• **Mapping** each assessment finding category (across the five products + suite) to a remediation procedure.

• **Applying** fixes **pilot-first**, **preview-gated**, with verification and re-score.

• **Producing** a change-tracked remediation evidence report proving closure.

What is out of scope (and where it lives)

• **Producing the findings** — the Security Optimization Assessments (read-only) generate the backlog this playbook consumes.

• **Deep product build-outs** — the product-specific onboarding/optimization solutions (e.g. *Defender XDR Unified Onboarding*, *Defender for Cloud CSPM & Secure Score Remediation*, *EOP/MDO mail hygiene*, *Cloud App DLP*) go deeper than a single finding-fix; the playbook points to them for large gaps.

• **Production change-approval, risk-acceptance, regulatory, and incident-response decisions** — the customer's change-management, security, compliance, and SOC functions.

Connections and authentication used

• **Microsoft Graph** (read for ingest/verify; **write scopes only where a specific remediation requires them**, e.g. policy configuration) and **Az** (for Defender for Cloud plan/coverage remediation). *Write scopes are requested only by the gated remediation step, never by ingest/verify/re-score.*

• The **Microsoft Defender portal** and **Azure portal** for the portal-driven remediations the scripts cannot fully automate.

• Roles: **read-only** (Security Reader / Reader) for ingest/verify/re-score; a **scoped admin** role (e.g. Security Administrator, or the specific product admin) **only** for the apply step, granted per the customer's change process.

The standard deployment model

> ***Assess*** *the finding (ingest from the report) →* ***Configure*** *the remediation and its pilot scope →* ***Pilot*** *the change (preview with* `-WhatIf`*/simulation, then apply to the pilot) →* ***Validate*** *by verifying the control and* ***re-scoring*** *to prove closure →* ***Expand*** *across scope in stages with rollback ready. Repeatable pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand****, every change* ***preview/****-*`WhatIf` ***first****.*

2. Requirements — Remediation Playbook for Defender Security Optimization Assessments

The category-wide prerequisites (the target tenant, assessor access, Graph/Az access, licensing, scope/baseline, output handling, remediation ownership) are documented once in `global-requirements-assessments.md` — read that first. This section lists only the prerequisites **specific to remediation**. Because this playbook **makes changes**, the prerequisites include change-control items the read-only assessments did not need. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. A completed assessment findings report to remediate

Description: A findings report from one of the Defender Security Optimization Assessments (SOAMCA / SOAMDC / SOAXDR / the endpoint/identity/office siblings), with findings prioritized by risk. **Why required:** The playbook remediates *specific findings*; without a report there is nothing to close (and no baseline to re-score against). **How to verify:** You hold a `Findings Report - <product> - Validate.json` (or equivalent) with ranked findings and their remediation pointers. **How to implement if missing:** Run the relevant Security Optimization Assessment first (read-only) to produce the backlog, then return here.

R2. Production change-approval process and a named change window

Description: The customer's change-management approval for the remediations, and an agreed change window for the pilot and the rollout. **Why required:** This playbook changes production configuration; **production change-approval is the customer's decision**, not the playbook's. Disruptive/response actions are approval-gated. **How to verify:** A change ticket / approval exists naming the findings to remediate, the pilot scope, and the window. **How to implement if missing:** Do not apply any change until approval exists. The preview (`-WhatIf`) and ingest steps are safe to run beforehand to prepare the change record.

R3. A named pilot scope with rollback / break-glass

Description: A small, named pilot scope for each remediation (e.g. a pilot user group, one subscription, a test policy) plus a documented rollback and a break-glass account preserved from any tightening. **Why required:** Pilot-first limits blast radius; rollback/break-glass ensures a bad change can be undone and never locks everyone out. **How to verify:** The pilot scope is defined; a break-glass account is excluded from any access/CA/session tightening; the rollback step for each remediation is written down. **How to implement if missing:** Create a pilot group and confirm a break-glass account is excluded before applying anything disruptive.

R4. Least-privilege read roles for ingest/verify/re-score

Description: Security Reader / Reader (read-only) for the ingest, verify, and re-score steps. **Why required:** Most of the loop is read-only; only the apply step needs write. Keeping read and write separated proves the non-apply steps changed nothing. **How to verify:** You can run `SOAREM-Discovery.ps1` (ingest) and `SOAREM-Rescore.ps1` with read-only roles. **How to implement if missing:** Assign Security Reader / Reader for the read steps.

R5. A scoped admin role for the apply step ONLY

Description: A **scoped admin** role sufficient for the specific remediation (e.g. Security Administrator for policy changes, the relevant product admin, or Contributor at an Azure scope for plan enablement) — granted per the customer's change process, ideally **just-in-time (PIM)**. **Why required:** The `-WhatIf`-gated remediation step needs write access to apply a change; it should be scoped and time-bound, not standing Global Admin. **How to verify:** The apply role is granted for the change window and removed after; `SOAREM-Remediate.ps1 -Apply` can perform the specific change. **How to implement if missing:** Request the minimal scoped role for the window via PIM; do not use standing Global Admin. Never grant write roles to the ingest/verify/re-score steps.

R6. Write scopes for the specific remediation (only where required)

Description: The Graph/Az write scopes a *specific* remediation needs (e.g. a policy-write scope) — requested only by `SOAREM-Remediate.ps1`, never by ingest/verify/re-score. **Why required:** Applying a configuration change requires the matching write scope; keeping it isolated to the apply script preserves the read-only guarantee of the rest. **How to verify:** `SOAREM-Remediate.ps1` in `-WhatIf` mode lists the scope it *would* need; it only requests write when you pass `-Apply`. **How to implement if missing:** Consent the specific write scope for the apply step per the change process; keep it minimal.

R7. A secure location for the evidence report

Description: A restricted place to store the change-tracked remediation evidence report (before/after scores, what changed, when, by whom). **Why required:** The evidence report is both sensitive and auditable (Global Requirements G7). **How to verify:** A restricted location exists for `SOAREM-*.json` outputs and the evidence report. **How to implement if missing:** Create a restricted location and point the scripts at it.

Global Requirements — Assessments (category-level)

These requirements apply to every solution in the Assessments category (Security Optimization Assessments, remediation playbooks, and cross-cutting security assessments). Published once at the category level so listings stay small. Confirm these before running any assessment. Each entry: Description / Why / Verify / Implement-if-missing.

> *An assessment observes and scores; it does not change your tenant. Every assessment script here is **read-only** (the paired remediation work is separate and pilot-first). Scope, risk-acceptance, and remediation-priority decisions belong to the customer's security/compliance owners — the assessment supplies the evidence, not the authorization.*

G1. The workload/tenant being assessed

Description: An active Microsoft 365 / Entra tenant and/or Azure subscription(s) hosting the products in scope. Why: There is nothing to assess without the target environment. Verify: Sign in to the relevant portal(s) (`security.microsoft.com`, `portal.azure.com`) as an administrator. Implement: Ensure the target tenant/subscription exists and the products under assessment are deployed (even partially — assessing coverage gaps is the point).

G2. Read-only assessor access (least privilege)

Description: A read-only role sufficient to enumerate configuration — e.g. Security Reader, Global Reader, and/or Reader at the Azure scope. Why: An assessment must never require write access; read-only proves it changes nothing. Verify: Entra -> `Roles and administrators`; confirm Security Reader / Global Reader (and Azure Reader where cloud resources are assessed). Implement: Assign the read-only roles; do not grant write/admin roles for the assessment itself.

G3. Microsoft Graph / Az / management API access

Description: Microsoft Graph (Security), Azure Resource Manager, and product-specific management APIs reachable with the assessor identity. Why: The assessment scripts collect configuration and posture via these APIs. Verify: `Connect-MgGraph` (read scopes) and `Connect-AzAccount` succeed; the assessment discovery script runs. *Verify current API/permission scopes.* Implement: Consent the read-only Graph scopes; ensure network access to the Microsoft endpoints.

G4. Licensing for the products in scope

Description: The licenses/plans for the products being assessed (e.g. Defender for Endpoint/Identity/Office 365/Cloud Apps plans, Defender for Cloud plans). Why: Some posture/configuration is only present (and only assessable) when the plan is enabled; the assessment flags plan gaps too. Verify: Confirm the relevant plans in the admin/billing portal. *Verify current plan/feature mapping.* Implement: Note unlicensed products as findings (a coverage gap) rather than skipping them silently.

G5. Scope definition and a point-in-time baseline

Description: A defined assessment scope (which products, subscriptions, and users) and the understanding that results are a point-in-time snapshot. Why: Scope keeps the assessment bounded and the score comparable; posture drifts, so results are dated. Verify: The scope is documented; the assessment records the run date. Implement: Agree the scope before running; re-run periodically to trend

the score.

G6. Best-practice reference baseline

Description: The best-practice / benchmark the assessment scores against (Microsoft security baselines, Secure Score, CIS/Zero-Trust as applicable). Why: A score is only meaningful against a reference; the assessment states which baseline it uses. Verify: The reference baseline is named in the findings. *Baselines evolve — verify against current Microsoft guidance.* Implement: Use the documented baseline; note where organizational policy intentionally deviates.

G7. Output handling and confidentiality

Description: A place to store the findings report and an understanding that it may contain sensitive posture detail. Why: Assessment output is a roadmap of your weaknesses; it must be handled as sensitive. Verify: A secure destination for the report exists; access is restricted. Implement: Store findings securely; share on a need-to-know basis. *Risk-acceptance of any finding is the customer's decision.*

G8. Remediation ownership (for the paired remediation)

Description: Named owners for acting on findings (the remediation is separate, pilot-first work). Why: An assessment only creates value when someone owns closing the gaps. Verify: Remediation owners are identified; the Remediation Playbook solution is available for the fixes. Implement: Assign owners; use the Remediation Playbook (pilot-first, -WhatIf-gated) to close findings and re-score.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Remediation Playbook for Defender Security Optimization Assessments

This is the exact, ordered procedure to remediate an assessment finding end to end — **pilot-first, preview-gated, verified, re-scored**. Unlike the assessments, this playbook **makes changes**; every change is previewed with `-WhatIf` / simulation, applied to a pilot scope first, and carries a rollback.

Naming convention. Every artifact you create uses `<Purpose> – <Scope> – <Stage>`. Use these exact names in the scripts, testing, and the evidence report:

- Remediation plan record → `Remediation Plan – Defender SOA – Assess`
- Pilot scope definition → `Pilot Scope – Defender SOA – Pilot`
- Pre-change score baseline → `Rescore Baseline – Defender SOA – Assess`
- Post-change re-score → `Rescore Result – Defender SOA – Validate`
- Remediation evidence report → `Evidence Report – Defender SOA – Validate`

Placeholders use `contoso.onmicrosoft.com` and a pilot group. Substitute yours. Confirm the prerequisites in Section 2 (including change-approval R2 and pilot/rollback R3) before applying anything.

The finding-to-remediation map

Each finding category maps to a remediation and its gating. **All are pilot-first and preview-gated; disruptive/response actions are additionally approval-gated.**

Product	Finding category	Remediation (pilot-first, `-WhatIf`/simulate first)	Extra gate
Cloud Apps	Connector/discovery coverage	Connect the missing app / enable continuous log upload or MDE integration	—
Cloud Apps	App-governance/OAuth	Enable app governance; create OAuth-app policy for risky consent	approval (may block apps)
Cloud Apps	Session/access policies	Create Conditional Access App Control session/access policy in **monitor** first	approval (can block users)
Cloud Apps	Alert hygiene	Tune alert policy severity/scope/exclusions; retire duplicates	—
Cloud	Secure Score recommendations	Apply the recommendation's Fix on the pilot subscription	—
Cloud	Plan coverage	Enable the workload plan on the pilot subscription	approval (has cost)
Cloud	Regulatory compliance	Map the required standard to the compliance dashboard	—
Cloud	Agent/agentless coverage	Enable agentless scanning / onboard the agent on the pilot scope	approval (has cost)
Endpoint	Onboarding/ASR/EDR	Onboard pilot devices; enable ASR rules in **audit** first	approval (audit→block)
Identity	Sensor/health/detections	Deploy/repair sensors on a pilot DC; resolve health alerts	—
Office 365	Anti-phish/Safe Links/Attachments	Apply preset security policy / tighten policy on a pilot group	approval (mail impact)

Product	Finding category	Remediation (pilot-first, `-WhatIf`/simulate first)	Extra gate
Suite/XDR	Cross-pillar coverage	Onboard the dark pillar (points to its onboarding solution)	approval
Suite/XDR	Unified RBAC	Activate unified RBAC; assign least-privilege roles to a pilot	approval (access change)
Suite/XDR	Incident correlation	Ensure pillars feed signal (downstream of coverage)	—

Stage 1 — Assess (ingest the finding and record the plan)

Step 1.1 — Ingest the findings (read-only)

1. Open PowerShell in `scripts/`. Connect read-only:

```
Connect-MgGraph -Scopes "SecurityEvents.Read.All","Directory.Read.All"
```

1. Ingest the assessment's findings report:

```
.\SOAREM-Discovery.ps1 -FindingsReportPath ".\Findings Report - Cloud Apps - Validate.json" -OutputPath
```

1. This reads the ranked findings, maps each to its remediation (per the map above), and writes `Remediation Plan – Defender SOA – Assess`. Read-only.

Step 1.2 — Capture the pre-change score baseline

1. Re-run the relevant assessment score (or copy its last score) and save it as `Rescore Baseline – Defender SOA – Assess` — this is the "before" you will prove you improved.

Stage 2 — Configure (plan the fix and the pilot)

Step 2.1 — Define the pilot scope

1. Record the pilot scope in `Pilot Scope – Defender SOA – Pilot`: the pilot group/subscription/policy, the break-glass exclusion, the change window, and the rollback step for this finding (R3).

2. Confirm change-approval (R2) covers this finding and scope.

Step 2.2 — Preview the change (`-WhatIf` — touches nothing)

1. Run the remediation in **preview** mode (the default — no `-Apply`):

```
.\SOAREM-Remediate.ps1 -RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json" -FindingI
```

1. Read exactly what *would* change. Nothing is applied. If it looks wrong, stop and adjust the plan.

Stage 3 — Pilot (apply to the pilot scope, gated)

Step 3.1 — Apply to the pilot only

1. With approval in hand and the preview reviewed, apply to the **pilot scope**:

```
.\SOAREM-Remediate.ps1 -RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json" -FindingI
```

1. The script requires the explicit `-Apply` switch and, for **approval-gated** categories (blocking policies, response actions, paid plans), an additional `-ConfirmDisruptive` acknowledgement. For monitor-capable controls (session/access, ASR), it applies in **monitor/audit** mode first.

2. Keep the break-glass account excluded (R3).

Stage 4 — Validate (verify the control, then re-score)

Step 4.1 — Verify the specific control

1. Run the post-change validation:

```
.\SOAREM-Validate.ps1 -RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json" -FindingId
```

1. This confirms the specific control is now correct on the pilot scope (e.g. the OAuth policy exists, the plan is on, the session policy is in monitor). Read-only.

Step 4.2 — Re-score to prove closure

1. Re-run the relevant assessment score and compare to the pre-change baseline:

```
.\SOAREM-Rescore.ps1 -BaselinePath ".\Rescore Baseline - Defender SOA - Assess.json" -OutputPath ".\Res
```

1. The re-score is the **objective proof** the finding closed — the domain/overall score should rise. If it did not move, the fix did not take (see Troubleshooting).

Stage 5 — Expand (stage the rollout and evidence it)

Step 5.1 — Expand the change in stages

1. With the pilot verified and re-scored, expand the change across scope in stages — for monitor-capable controls, move from **monitor/audit** to **enforce** only after the pilot is clean.

2. Keep the rollback ready at each stage; keep break-glass excluded.

Step 5.2 — Produce the evidence report

1. Generate the change-tracked evidence report:

```
.\SOAREM-Report.ps1 -RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json" -ValidationP
```

1. `Evidence Report – Defender SOA – Validate` records each finding, what changed, when, by whom, the pilot→enforce stage, and the before/after score — the auditable proof the backlog closed. Store it securely (R7).

Every change above is preview/`-WhatIf` first, pilot-first, and reversible. If any remediation cannot be previewed, do not apply it — stop and review.

5. Testing — Remediation Playbook for Defender Security Optimization Assessments

These procedures validate that remediation is **safe, effective, and reversible** — that preview truly changes nothing, the apply is scoped to the pilot, the control is verified, and the re-score proves closure. Each test has an explicit **pass/fail gate**. Reference the scripts by the names used in Section 3.

Test 1 — Ingest is read-only and maps findings

Procedure: Run `.\SOAREM-Discovery.ps1 -FindingsReportPath ... -OutputPath ...`. **Pass:** It writes `Remediation Plan - Defender SOA - Assess.json` with each finding mapped to a remediation and its gate; no write scope was requested. **Fail:** A write scope prompt, or a finding with no mapped remediation — re-check the findings report and the map.

Test 2 — Preview (`-WhatIf`) changes nothing

Procedure: Run `.\SOAREM-Remediate.ps1 ... -WhatIf` for a finding. **Pass:** The script prints exactly what *would* change and confirms **no change was made**; a subsequent read of the target shows the old state unchanged. **Fail:** Any actual change occurs in `-WhatIf` mode — a critical defect; stop and do not use the apply path until fixed.

Test 3 — Apply requires the explicit switch (and disruptive confirmation)

Procedure: Run the remediate script **without** `-Apply`, then **with** `-Apply` on a monitor-capable control; then attempt an approval-gated (disruptive) category without `-ConfirmDisruptive`. **Pass:** Without `-Apply` nothing changes; with `-Apply` the pilot-scope change is made; a disruptive category **refuses** to enforce without `-ConfirmDisruptive` (and applies in monitor/audit where supported). **Fail:** The script applies without `-Apply`, or enforces a disruptive control without the confirmation — a gating defect.

Test 4 — Apply is scoped to the pilot only

Procedure: After a pilot apply, confirm the change affects **only** the named pilot scope and the break-glass account is excluded. **Pass:** Only the pilot group/subscription/policy changed; break-glass is untouched. **Fail:** The change hit production or the break-glass account — roll back immediately (Section 6) and re-scope.

Test 5 — Verification confirms the specific control

Procedure: Run `.\SOAREM-Validate.ps1 -FindingId ...`. **Pass:** It confirms the specific control is now correct on the pilot (e.g. policy present, plan on, session policy in monitor), read-only. **Fail:** Verification says the control is not in place though you applied it — the apply did not take (Troubleshooting).

Test 6 — Re-score proves closure

Procedure: Run `.\SOAREM-Rescore.ps1 -BaselinePath ...` and compare to the pre-change baseline. **Pass:** The relevant domain/overall score **rises** versus the baseline, proving the finding closed (allowing for warm-up). **Fail:** The score does not move after a verified change — allow warm-up and re-run; if still flat, the change may not affect the scored control.

Test 7 — Rollback works

Procedure: On the pilot, exercise the documented rollback for the applied remediation. **Pass:** The change is cleanly reverted and the target returns to its prior state. **Fail:** Rollback leaves a partial/broken state — do not expand the rollout until rollback is proven.

Test 8 — Evidence report is complete and traceable

Procedure: Run `.\SOAREM-Report.ps1 ...` and open the evidence report. **Pass:** Each remediated finding shows what changed, when, by whom, the stage (monitor→enforce), and the before/after score. **Fail:** A remediation lacks change-tracking or before/after evidence — the closure is not auditable.

Exit criteria

For each finding: preview verified harmless, apply scoped to pilot with approval, control verified, re-score risen, rollback proven, and the evidence report complete. Only then expand to enforce/broad scope.

6. Troubleshooting — Remediation Playbook for Defender Security Optimization Assessments

Common failures when remediating, in **symptom** → **cause** → **fix** form. Because this playbook changes configuration, several fixes involve **rollback** — always keep the pilot scope small and break-glass excluded.

`SOAREM-Discovery.ps1` can't map a finding

Symptom: Ingest leaves a finding with no mapped remediation. **Cause:** The findings report is from an unrecognized format/product, or the finding category is not in the map. **Fix:** Confirm the findings report is a current SOA report (R1). If the category is genuinely new, treat it as a manual remediation and record it; do not force an unrelated fix.

`-WhatIf` seems to make a change

Symptom: You suspect the preview altered something. **Cause:** This must never happen — `-WhatIf` is preview-only. **Fix:** Stop using the apply path. Re-read the target to confirm state, and treat any real change under `-WhatIf` as a critical defect (Testing Test 2). Do not proceed until preview is proven harmless.

Apply fails with an authorization error

Symptom: `SOAREM-Remediate.ps1 -Apply` errors on permissions. **Cause:** The scoped admin role (R5) or the specific write scope (R6) is not granted for the change window. **Fix:** Request the minimal scoped role (via PIM) and consent the specific write scope for the apply step. Never fall back to standing Global Admin.

The remediation refuses to enforce

Symptom: An approval-gated category (blocking policy, response action, paid plan) will not enforce. **Cause:** By design — it requires `-ConfirmDisruptive` and applies in monitor/audit first. **Fix:** Obtain change-approval (R2), review the monitor-mode results, then pass `-ConfirmDisruptive` to move to

enforce. This gate is intentional.

The change hit more than the pilot scope

Symptom: Users/resources outside the pilot were affected. **Cause:** The pilot scope was too broad, or a policy applied tenant-wide instead of to the pilot group. **Fix: Roll back immediately** using the documented rollback (R3). Re-scope to a tighter pilot group and re-run in `-WhatIf` before re-applying. Confirm break-glass was excluded.

Verification says the control isn't in place

Symptom: `SOAREM-Validate.ps1` reports the control missing after an apply. **Cause 1:** The apply did not complete (transient error), or targeted a different scope than verified. **Cause 2:** Propagation delay. **Fix:** Confirm the apply's scope matches the verify's scope. Allow propagation and re-run verify. If still missing, re-apply after reviewing the preview.

Re-score doesn't move after a verified change

Symptom: `SOAREM-Rescore.ps1` shows no score improvement though the control is verified. **Cause 1:** Warm-up — Secure Score / Defender assessments take time to reflect changes. **Cause 2:** The fixed control isn't the one the score measures (partial match to the finding). **Fix:** Allow a warm-up window and re-run the re-score (Global Requirements G5). If still flat, re-check that the remediation targets the exact scored control; the finding may need a deeper solution.

Rollback leaves a partial state

Symptom: Reverting the change doesn't fully restore the prior state. **Cause:** The rollback step was incomplete, or the change had dependent side effects. **Fix:** Do not expand the rollout until rollback is proven clean on the pilot (Testing Test 7). Restore from the documented prior configuration; for policy changes, keep the pre-change export to re-import.

Locked out after a tightening change

Symptom: Admins/users can't access after an access/CA/session/RBAC change. **Cause:** Break-glass was not excluded, or the change was broader than intended. **Fix:** Use the **break-glass account** (preserved per R3) to sign in and roll back. This is exactly why break-glass exclusion is mandatory before any tightening.

7. Best Practices — Remediation Playbook for Defender Security Optimization Assessments

Field-proven guidance for remediating safely and proving the gaps actually closed.

Preview before you touch anything

- Always run the remediation in `-WhatIf` **/ simulation** first and read exactly what would change. If a remediation cannot be previewed, do not apply it. Preview is free and catches surprises before they hit

users.

Pilot before you enforce

• Apply to a small, **named pilot scope** first (pilot group, one subscription, a test policy). For monitor-capable controls (Conditional Access App Control session/access, ASR rules, DLP), run in **monitor/audit** mode, watch, then move to **enforce** only when the pilot is clean.

• Keep a **break-glass account excluded** from every access/CA/session/RBAC tightening — this is the single most important habit; it prevents a bad change from locking everyone out.

Change under approval, on the customer's authority

• **Production change-approval, risk-acceptance, and remediation priority are the customer's decisions.** The playbook supplies the technical fix; it does not authorize changing production. Disruptive controls and response actions are **approval-gated** — require the explicit confirmation.

• Grant the apply role **just-in-time (PIM)**, scoped and time-bound — never remediate from standing Global Admin.

Re-score is the proof, not the apply

• A change applied is not a gap closed. **Re-run the relevant assessment score** and show the domain/overall score rise versus the pre-change baseline. Allow warm-up — Secure Score and Defender assessments take time to reflect a change.

• If a verified change does not move the score, the change may not affect the scored control — investigate before claiming closure.

Keep read and write separated

• Ingest, verify, and re-score run **read-only** (Security Reader / Reader). Only `SOAREM-Remediate.ps1 -Apply` needs write. This separation preserves the audit story: everything except the deliberate apply changed nothing.

Track everything for audit

• Record what changed, when, by whom, the monitor→enforce stage, and the before/after score in the **evidence report**. Store it securely (R7 / Global Requirements G7). This is what proves the backlog closed.

Remediate by risk, in stages

• Work the highest-risk findings first (the report already ranks them), but remediate **one category at a time** through the full loop so you can attribute a score movement to a specific fix.

Know when to hand off to a deeper solution

• A single finding-fix is not a product build-out. If a finding is really a **whole dark pillar** or a large gap, point to the deeper solution (e.g. *Defender XDR Unified Onboarding*, *Defender for Cloud CSPM &

Secure Score Remediation*, *EOP/MDO mail hygiene*, *Cloud App DLP*) rather than forcing it through a single remediation.

Where to go next (adjacent catalog areas)

• **Security Optimization Assessments** (Cloud Apps / Cloud / Suite-XDR / Endpoint / Identity / Office 365) — the read-only assessments that produce the backlog this playbook closes; re-run them to trend.

• **The product onboarding/optimization solutions** — for large gaps a single remediation cannot cover.

• **SOC Onboarding (Sentinel + Defender XDR)** and **Threat Hunting (KQL)** — to operationalize monitoring once the posture is remediated.

8. Deliverables Summary — Remediation Playbook for Defender Security Optimization Assessments

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box — turn assessment findings into closed, re-scored gaps, pilot-first and preview-gated
1. Solution Overview	The remediation model, the finding-to-remediation map across the five Defender products + suite, and the gating/verification/re-score approach
2. Requirements Guide	Solution-specific prerequisites (including change-approval, pilot/rollback, and read-vs-write role separation) with Verify / Implement-if-missing
Global Requirements (Assessments)	Category-wide prerequisites, published once
3. Implementation Guide	The finding-to-remediation map and exact, ordered steps to remediate a finding — Assess → Configure → Pilot → Validate → Expand — preview-gated, pilot-first, verified, re-scored
5. Testing Guide	Eight validation tests with pass/fail gates proving remediation is safe, effective, and reversible
6. Troubleshooting Guide	Common remediation failures and fixes, including rollback
7. Best Practices	Field-proven guidance and where to go next

PowerShell scripts (Microsoft Graph + Az)

Script	What it does	Mode
`SOAREM-Discovery.ps1`	Ingests the assessment findings report (read-only), maps each finding to its remediation and gate, writes the remediation plan.	Read-only
`SOAREM-Remediate.ps1`	The gated change helper — **defaults to `-WhatIf` preview** (touches nothing); applies to the named pilot scope only with an explicit `-Apply`, and refuses to enforce approval-gated/disruptive categories without `-ConfirmDisruptive` (monitor/audit first where supported).	`-WhatIf` **default; write only on** `-Apply`
`SOAREM-Validate.ps1`	Verifies the specific control is now correct on the pilot scope after apply.	Read-only
`SOAREM-Rescore.ps1`	Re-runs the relevant assessment score and compares to the pre-change baseline — the objective proof the finding closed.	Read-only
`SOAREM-Report.ps1`	Produces the change-tracked remediation evidence report (finding, what changed, when, by whom, stage, before/after score).	Read-only

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **PowerShell scripts** (Microsoft Graph + Az) in a single ZIP — every change `-WhatIf`**/simulate-gated**, pilot-first, with verification and re-score.
- A repeatable process following **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — preview-gated, pilot-first, verified, re-scored, with rollback and an auditable evidence report.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

The assessment backlog **actually closed** — each finding remediated pilot-first with the change previewed before it is applied, verified, and re-scored to prove closure, with change-tracked evidence. The companion that makes the Security Optimization Assessments worth running.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SOAREM-Discovery.ps1

Purpose: Ingests a findings report produced by any Defender Security Optimization Assessment (SOAMCA / SOAMDC / SOAXDR / endpoint / identity / office). For each finding it looks up the remediation procedure and its gating (pilot-first always; some categories additionally approval-gated / disruptive), and writes the remediation plan (`Remediation Plan - Defender SOA - Assess`). READ-ONLY — reads the report and emits a plan; changes nothing in the tenant. HOW TO RUN Connect-MgGraph -Scopes "SecurityEvents.Read.All","Directory.Read.All" # read-only powershell -ExecutionPolicy Bypass -File .\SOAREM-Discovery.ps1 `-FindingsReportPath ".\Findings Report - Cloud Apps - Validate.json"` -OutputPath ".\Remediation Plan - Defender SOA - Assess.json" EXPECTED RESULTS A remediation plan JSON: each finding with its mapped remediation, gate, and pilot-first note. .NOTES White-label. Read-only. Does NOT apply any change — SOAREM-Remediate.ps1 performs the gated apply. Production change-approval and priority are the customer's decisions. > [CmdletBinding()] param([Parameter(Mandatory)][string]$FindingsReportPath, [string]$OutputPath = ".\Remediation Plan - Defender SOA - Assess.json")

How to run: Connect-MgGraph -Scopes "SecurityEvents.Read.All","Directory.Read.All" # read-only powershell -ExecutionPolicy Bypass -File .\SOAREM-Discovery.ps1 `-FindingsReportPath ".\Findings Report - Cloud Apps - Validate.json"` -OutputPath ".\Remediation Plan - Defender SOA - Assess.json" EXPECTED RESULTS A remediation plan JSON: each finding with its mapped remediation, gate, and pilot-first note. .NOTES White-label. Read-only. Does NOT apply any change — SOAREM-Remediate.ps1 performs the gated apply. Production change-approval and priority are the customer's decisions. > [CmdletBinding()] param([Parameter(Mandatory)][string]$FindingsReportPath, [string]$OutputPath = ".\Remediation Plan - Defender SOA - Assess.json")

SOAREM-Remediate.ps1

Purpose: Applies a single mapped remediation from the remediation plan to a PILOT scope, pilot-first and preview-gated. Safety model (do not weaken): Default = PREVIEW. With no -Apply, the script only prints what WOULD change. No writes. -Apply is required to make any change, and the change is confined to -PilotScope. Approval-gated / disruptive categories (blocking policies, response actions, paid plans, RBAC/onboarding) additionally require -ConfirmDisruptive, and apply in MONITOR/AUDIT mode first where the control supports it. A break-glass account must be excluded from any access/CA/session/RBAC tightening (caller's responsibility per Requirements R3) — the script warns if -BreakGlassUpn is not supplied. The per-finding write calls are provided as GUARDED STUBS (commented real calls) so this white-label template never performs an unexpected change; an operator fills in / confirms the exact call for their finding inside the gated block before enabling it. HOW TO RUN Preview (safe, default) — touches nothing: .\SOAREM-Remediate.ps1 -RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json" `-FindingId "<finding id>" -PilotScope "Pilot Group - Defender SOA" -WhatIf Apply to pilot (non-disruptive category): .\SOAREM-Remediate.ps1 -RemediationPlanPath "..." -FindingId "<id>" -PilotScope "..." -Apply Apply an approval-gated/disruptive category (monitor/audit first): .\SOAREM-Remediate.ps1 -RemediationPlanPath "..." -FindingId "<id>"`

-PilotScope "..." -Apply -ConfirmDisruptive -BreakGlassUpn "breakglass@contoso.onmicrosoft.com" EXPECTED RESULTS Preview: a printed description of the intended change; nothing altered. Apply: the change made ONLY within the pilot scope, in monitor/audit mode where supported, with a change record returned for the evidence report. .NOTES White-label. WRITE occurs ONLY on -Apply. Production change-approval is the customer's decision. Keep this gating intact — weakening it defeats the purpose. Verify current cmdlets/scopes. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$RemediationPlanPath, [Parameter(Mandatory)][string]$FindingId, [Parameter(Mandatory)][string]$PilotScope, [switch]$Apply, [switch]$ConfirmDisruptive, [string]$BreakGlassUpn, [string]$OutputPath = ".\SOAREM-change-record.json")

How to run: Preview (safe, default) — touches nothing: .\SOAREM-Remediate.ps1 -RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json" -FindingId "<finding id>" -PilotScope "Pilot Group - Defender SOA" -WhatIf Apply to pilot (non-disruptive category): .\SOAREM-Remediate.ps1 -RemediationPlanPath "..." -FindingId "<id>" -PilotScope "..." -Apply Apply an approval-gated/disruptive category (monitor/audit first): .\SOAREM-Remediate.ps1 -RemediationPlanPath "..." -FindingId "<id>" -PilotScope "..." -Apply -ConfirmDisruptive -BreakGlassUpn "breakglass@contoso.onmicrosoft.com" EXPECTED RESULTS Preview: a printed description of the intended change; nothing altered. Apply: the change made ONLY within the pilot scope, in monitor/audit mode where supported, with a change record returned for the evidence report. .NOTES White-label. WRITE occurs ONLY on -Apply. Production change-approval is the customer's decision. Keep this gating intact — weakening it defeats the purpose. Verify current cmdlets/scopes. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$RemediationPlanPath, [Parameter(Mandatory)][string]$FindingId, [Parameter(Mandatory)][string]$PilotScope, [switch]$Apply, [switch]$ConfirmDisruptive, [string]$BreakGlassUpn, [string]$OutputPath = ".\SOAREM-change-record.json")

SOAREM-Report.ps1

Purpose: Combines the remediation plan, the change record(s) from SOAREM-Remediate.ps1, the post-change verification, and the re-score into a single auditable evidence report proving each finding was remediated pilot-first, verified, and re-scored. READ-ONLY — it assembles evidence; it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOAREM-Report.ps1 -RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json" -ChangeRecordPath ".\SOAREM-change-record.json" -ValidationPath ".\SOAREM-validation.json" -RescorePath ".\Rescore Result - Defender SOA - Validate.json" ` -OutputPath ".\Evidence Report - Defender SOA - Validate.json" -ChangeRecordPath accepts a single file or a folder of change records. EXPECTED RESULTS A remediation evidence report (JSON + console summary): per-finding change-tracking and the before/after score — the auditable proof the backlog closed. .NOTES White-label. Read-only. Store the evidence report securely (Requirements R7). Production change-approval and risk-acceptance remain the customer's decisions. > [CmdletBinding()] param([Parameter(Mandatory)][string]$RemediationPlanPath, [string]$ChangeRecordPath, [string]$ValidationPath, [string]$RescorePath, [string]$OutputPath = ".\Evidence Report - Defender SOA - Validate.json")

How to run: powershell -ExecutionPolicy Bypass -File .\SOAREM-Report.ps1 -RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json" -ChangeRecordPath ".\SOAREM-change-record.json" -ValidationPath ".\SOAREM-validation.json" -RescorePath

".\Rescore Result - Defender SOA - Validate.json" ` -OutputPath ".\Evidence Report - Defender SOA - Validate.json" -ChangeRecordPath accepts a single file or a folder of change records. EXPECTED RESULTS A remediation evidence report (JSON + console summary): per-finding change-tracking and the before/after score — the auditable proof the backlog closed. .NOTES White-label. Read-only. Store the evidence report securely (Requirements R7). Production change-approval and risk-acceptance remain the customer's decisions. > [CmdletBinding()] param([Parameter(Mandatory)][string]$RemediationPlanPath, [string]$ChangeRecordPath, [string]$ValidationPath, [string]$RescorePath, [string]$OutputPath = ".\Evidence Report - Defender SOA - Validate.json")

SOAREM-Rescore.ps1

Purpose: The objective proof of closure. Takes a fresh posture-score file (produced by re-running the relevant assessment's *-Score.ps1 after the change) and compares it to the pre-change baseline captured before remediation. Reports the per-domain and overall DELTA — a rise proves the fix moved the needle. READ-ONLY — it compares scores; it changes nothing. HOW TO RUN 1) Re-run the relevant assessment score (e.g. SOAMCA-Score.ps1) AFTER the change, then: powershell -ExecutionPolicy Bypass -File .\SOAREM-Rescore.ps1 `-BaselinePath ".\Rescore Baseline - Defender SOA - Assess.json"` -NewScorePath ".\Posture Score - Cloud Apps - Pilot (post-change).json" ` -OutputPath ".\Rescore Result - Defender SOA - Validate.json" EXPECTED RESULTS A delta report: baseline vs new per-domain and overall scores, with the improvement (or note if flat — allow warm-up). This is the evidence the finding closed. .NOTES White-label. Read-only. Secure Score / Defender assessments take time to reflect a change — allow a warm-up window before trusting a flat re-score. A verified control with a flat score may not be the exact scored control; investigate before claiming closure. > [CmdletBinding()] param([Parameter(Mandatory)][string]$BaselinePath, [Parameter(Mandatory)][string]$NewScorePath, [string]$OutputPath = ".\Rescore Result - Defender SOA - Validate.json")

How to run: 1) Re-run the relevant assessment score (e.g. SOAMCA-Score.ps1) AFTER the change, then: powershell -ExecutionPolicy Bypass -File .\SOAREM-Rescore.ps1 `-BaselinePath ".\Rescore Baseline - Defender SOA - Assess.json"` -NewScorePath ".\Posture Score - Cloud Apps - Pilot (post-change).json" ` -OutputPath ".\Rescore Result - Defender SOA - Validate.json" EXPECTED RESULTS A delta report: baseline vs new per-domain and overall scores, with the improvement (or note if flat — allow warm-up). This is the evidence the finding closed. .NOTES White-label. Read-only. Secure Score / Defender assessments take time to reflect a change — allow a warm-up window before trusting a flat re-score. A verified control with a flat score may not be the exact scored control; investigate before claiming closure. > [CmdletBinding()] param([Parameter(Mandatory)][string]$BaselinePath, [Parameter(Mandatory)][string]$NewScorePath, [string]$OutputPath = ".\Rescore Result - Defender SOA - Validate.json")

SOAREM-Validate.ps1

Purpose: After SOAREM-Remediate.ps1 applies a change to the pilot scope, this script confirms the SPECIFIC control the finding was about is now in the expected state (e.g. the OAuth policy exists, the workload plan is on, the session policy is present in monitor). READ-ONLY — it checks state and reports Pass/Gap; it changes nothing. HOW TO RUN Connect-MgGraph -Scopes "SecurityEvents.Read.All","Directory.Read.All" # read-only powershell -ExecutionPolicy Bypass -File .\SOAREM-Validate.ps1 `-RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json"` -FindingId "<finding id>" -PilotScope "Pilot Group - Defender SOA" ` -OutputPath ".\SOAREM-validation.json" EXPECTED RESULTS A Pass/Gap verdict for the specific remediated control on the pilot scope, with evidence. .NOTES White-label. Read-only. Verification confirms the control is in

place; the re-score (SOAREM-Rescore.ps1) proves the finding closed. The state checks are provided as guarded stubs to be completed for the specific finding; verify current cmdlets. > [CmdletBinding()] param([Parameter(Mandatory)][string]$RemediationPlanPath, [Parameter(Mandatory)][string]$FindingId, [string]$PilotScope, [string]$OutputPath = ".\SOAREM-validation.json")

How to run: Connect-MgGraph -Scopes "SecurityEvents.Read.All","Directory.Read.All" # read-only powershell -ExecutionPolicy Bypass -File .\SOAREM-Validate.ps1 `-RemediationPlanPath ".\Remediation Plan - Defender SOA - Assess.json"` -FindingId "<finding id>" -PilotScope "Pilot Group - Defender SOA" ` -OutputPath ".\SOAREM-validation.json" EXPECTED RESULTS A Pass/Gap verdict for the specific remediated control on the pilot scope, with evidence. .NOTES White-label. Read-only. Verification confirms the control is in place; the re-score (SOAREM-Rescore.ps1) proves the finding closed. The state checks are provided as guarded stubs to be completed for the specific finding; verify current cmdlets. > [CmdletBinding()] param([Parameter(Mandatory)][string]$RemediationPlanPath, [Parameter(Mandatory)][string]$FindingId, [string]$PilotScope, [string]$OutputPath = ".\SOAREM-validation.json")